SHOWSHELF LAB · FOR THE AFTERNOON TEACHER

Teacher guide: Day 11

Students work through the steps in the Lab on their own. Each step has an explanation, a task, two hints and an answer button, so most of the time they can get unstuck alone. This guide gives you every task, a correct answer and the mistakes students usually make, so you can help quickly.

How to help: first ask them to read the "Not yet" message and the red Console line (if any), then open a hint. Only point at the answer after that. In the Lab, **Help › Teacher view** shows the answer and these notes on every step.

Day 11: Make it yours and publish

Personalise, download, put online, demo

The last day. This morning's lecture went back over the whole course. This afternoon, ShowShelf becomes **your** website: your name, your colours, a feature or two of your own, and an address on the internet that you can send to anyone.

The app is the one you finished on Day 9. Its files are read-only today (open the tabs above the code box to read them). You write in three new files instead:

- **site.js**: your site's name, a sentence about it, and your name (steps 1 and 2)
- **my-theme.css**: your colours, corners and font (steps 3 and 4)
- **extra.js**: new features that use the app's own functions (steps 5 to 7)

Then you download your website as one file, put it online (steps 8 and 9), and prepare a **2-minute demo** for the end of the day (step 10). Aim to finish step 9 with at least 30 minutes to spare.

What next? Keep practising a little every week. freeCodeCamp has free, step-by-step courses, and MDN Web Docs explains every HTML element, CSS property and JavaScript method. A great next project: build ShowShelf again from scratch with a different free API, for example one for music or for books.

Step 1. Tour your app ~20 min

Teaches: A website is several files loaded in order; settings live in one object (`SITE`).

TASK

1. Open the **index.html (whole page)** tab. Find the four `<script src="...">` lines at the bottom. Which file loads last?
2. Open **app.js**. Find the `state` object and the function `openShow`. You'll use both later.
3. Near the bottom of app.js, find **Your site's details**. Which fields of `SITE` does it use?
4. Back in **site.js**, change `name` to a name for your site, like "Couch Club" or "Ana's Shelf". Watch the logo.

COMMON MISTAKES

- Single quotes around a name with an apostrophe: `'Ana's Shelf'` ends the text too early. Use double quotes.
- Deleting the comma at the end of the line, or a quote.
- Typing in a read-only tab and wondering why nothing changes.

NOTE

Give students a few minutes to read app.js properly: steps 5 to 7 use `state`, `openShow`, `render` and `afterRender`. The order of the `<script>` lines explains why extra.js can use everything in app.js.

A CORRECT ANSWER

```
// ... unchanged ...

const SITE = {
  name: "Ana's Shelf",
  tagline: "Search real shows from TVmaze and keep your watchlist.",
  owner: "Your name",
// ... unchanged ...
```

Step 2. A tagline and your name ~10 min

Teaches: The tagline and "Made by" come from `SITE.tagline` and `SITE.owner`.

TASK

In `SITE`:

1. Change `tagline` to your own sentence about your site.
2. Change `owner` to your name.

Scroll down in the preview to see the footer.

COMMON MISTAKES

- Putting the comma inside the quotes: `owner: "Ana Novak,"` (it works, but the footer shows the comma).
- A double quote inside the tagline: `"The "best" shows"` ends the text early.

A CORRECT ANSWER

```
// ... unchanged ...
const SITE = {
  name: "Ana's Shelf",
  tagline: "The shows Ana loves, and the ones she's saving for later.",
  owner: "Ana Novak",
};
```

Step 3. Your colours ~20 min

Teaches: A later stylesheet overrides an earlier one; text needs enough contrast with its background.

TASK

Inside `:root { }`, under the comment, set:

1. `--color-accent`
2. `--color-bg`
3. `--color-surface`
4. one more colour of your choice from the list (for example `--color-accent-dark`, a darker shade of your accent)

The check also measures the contrast of your text and buttons.

COMMON MISTAKES

- Writing the new lines inside the comment, before the `*/`, so they're ignored.
- One dash instead of two: `-color-bg`.
- A dark `--color-bg` with the default dark text: the check reports low contrast.

NOTE

The contrast numbers follow the web accessibility guidelines (WCAG): 4.5 to 1 for normal text, 3 to 1 for large or bold text like buttons. Light yellow or pale accents usually fail the button test: that's a good conversation about why.

A CORRECT ANSWER

```
/* ... unchanged ... */
     And one that isn't a colour:
       --radius               how round the corners of the cards are */
  --color-accent: #0a7d5a;
  --color-accent-dark: #075c42;
  --color-bg: #f1f7f4;
  --color-surface: #ffffff;
}
```

Step 4. Corners and a font ~15 min

Teaches: `--radius` sets the corners; `font-family` on `body` changes the whole site's font.

TASK

1. Inside `:root { }`, add `--radius` with a new value, rounder (like `20px`) or sharper (like `0px`).
2. Under the `:root` rule, add a `body { }` rule with a `font-family` of your choice, ending with a general family (`serif` or `sans-serif`).

COMMON MISTAKES

- `--radius: 20;` without `px`: the browser ignores it.
- The `body` rule inside the `:root { }` brackets.
- A font name with spaces and no quotes, then a typo: the browser quietly uses the backup.

A CORRECT ANSWER

```
/* ... unchanged ... */
  --color-bg: #f1f7f4;
  --color-surface: #ffffff;
  --radius: 20px;
}

body {
  font-family: "Trebuchet MS", Arial, sans-serif;
}
```

Step 5. A "Surprise me" button ~20 min

Teaches: Add features from a separate file by using the app's `state` and functions.

TASK

In extra.js:

1. Create a `button` with the text "Surprise me" and the class `button-secondary`.
2. Append it to the hero: `document.querySelector("#home-view .hero")`.
3. On click: pick a random show from `state.shows` and call `openShow` with its `id`.

Try it after a search too: it should only pick from the shows you see.

COMMON MISTAKES

- `openShow(state.shows[index])`: the whole show instead of its `id`, so TVmaze can't find it.
- `Math.random() * 12`: works until a search loads fewer shows.
- Naming the button `button` or `grid`: "has already been declared".

NOTE

If a student's Console says "has already been declared", they reused a name from app.js. Any new name fixes it.

A CORRECT ANSWER

```
// extra.js: your own features. This file runs last, so everything in app.js is ready:
//   state          state.shows (the shows loaded now), state.myList, state.view ...
//   render()       draws the whole page from the state
//   openShow(id)   opens the show view for one show
//   toggleMyList(show), saveMyList(), homeView, grid ... and more.
// Open the app.js tab to see them all.

// ---------- Surprise me: open a random show ----------
const surpriseButton = document.createElement("button");
surpriseButton.className = "button-secondary";
surpriseButton.textContent = "Surprise me";
document.querySelector("#home-view .hero").append(surpriseButton);

surpriseButton.addEventListener("click", () => {
  if (state.shows.length === 0) {
    return;   // no shows loaded (yet)
  }
  const index = Math.floor(Math.random() * state.shows.length);
  openShow(state.shows[index].id);
});
```

Step 6. Top rated first ~15 min

Teaches: `sort((a, b) => b.rating - a.rating)` puts the highest first; change the state, then `render()`.

TASK

In extra.js, under your Surprise me code:

1. Create a button with the text "Top rated first" and the class `chip`, and append it to the toolbar: `document.querySelector(".toolbar")`.
2. On click: sort `state.shows` from the highest `rating` to the lowest, then call `render()`.

COMMON MISTAKES

- `a.rating - b.rating`: lowest first.
- Forgetting `render()`: the state is sorted but the page isn't redrawn.
- `state.shows = state.shows.sort(...)` works too, but isn't needed: `sort` changes the array itself.

A CORRECT ANSWER

```
// ... unchanged ...
  openShow(state.shows[index].id);
});

// ---------- Top rated first: sort the grid by rating ----------
const sortButton = document.createElement("button");
sortButton.className = "chip";
sortButton.textContent = "★ Top rated first";
document.querySelector(".toolbar").append(sortButton);

sortButton.addEventListener("click", () => {
  state.shows.sort((a, b) => b.rating - a.rating);   // highest rating first
  render();
});
```

Step 7. A personal touch ~15 min

Teaches: A hook (`afterRender`) lets extra code run at the right moment in someone else's code.

TASK

Choose **one** (or more):

1. **A welcome line** with your name in the hero, like "Welcome! This shelf belongs to Ana." (`SITE.owner` has your name.)
2. **A count line** in the hero, like "Shows loaded: 12", that changes after every search. Set its text in `afterRender`, from `state.shows.length`.
3. **A badge** on the cards of shows in My List. In `afterRender`, loop over the cards: a card's round button has `aria-pressed` "true" when its show is in My List (see `createListButton` in app.js).

COMMON MISTAKES

- Spelling the hook differently (`afterrender`, `afterRendering`): app.js never calls it.
- Setting the count once at the top: it says 0, because the shows haven't loaded yet.
- Two functions called `afterRender`: only the last one counts.

NOTE

Any of the three choices passes. The badge is the hardest; point stronger students at it.

A CORRECT ANSWER

```
// ... unchanged ...
  render();
});

// ---------- A personal touch: how many shows are loaded ----------
const countLine = document.createElement("p");
countLine.className = "meta";
document.querySelector("#home-view .hero").append(countLine);

// app.js calls afterRender at the end of every render(), so the line stays up to date
function afterRender() {
  countLine.textContent = "Shows loaded: " + state.shows.length;
}
```

Step 8. Download your website ~15 min

Teaches: A website can be one `index.html` file; `<meta name="description">` is the sentence search engines show.

TASK

1. In `SITE`, add a `description`: one sentence about your site, at least 20 characters.
2. Press **Check**. It makes sure the page has no errors and that your name and the site's name are set.
3. Press **Download my website**. The file goes to your **Downloads** folder. (If there's already an `index.html` there, your browser may call it `index (1).html`. Rename it to `index.html`.)
4. Open your Downloads folder and **double-click** `index.html`: your site opens in the browser, with real data and real posters from TVmaze. It needs internet now, and My List starts empty there.

COMMON MISTAKES

- Forgetting the comma at the end of the `owner` line before the new `description` line.
- Downloading twice and opening the old `index.html` instead of `index (1).html`.
- Opening the file without internet: the page says the shows could not be loaded (the Lab's sample data isn't in the file).

NOTE

Some browsers ask where to save; tell students to choose Downloads or the Desktop and remember where. If a student only sees the code when opening the file, it was opened in a text editor: right-click, Open with, and choose a browser.

A CORRECT ANSWER

```
// ... unchanged ...
  tagline: "The shows Ana loves, and the ones she's saving for later.",
  owner: "Ana Novak",
  description: "My TV show finder: search real shows, see their cast and seasons, and keep a
watchlist.",
};
```

Step 9. Put it online ~25 min

Teaches: Hosting puts your files on a web server; deploying is uploading them.

TASK

Option A: Netlify Drop

1. On your computer, make a new folder, for example `my-showshelf`, and move your `index.html` into it.
2. Open app.netlify.com/drop. If it asks you to sign up or log in, make a free account: you need one so your site stays online.
3. Drag the **folder** from your computer onto the page and drop it.
4. Wait a few seconds. Netlify shows your site's address, ending in `.netlify.app`. Open it.

Option B: GitHub Pages

1. Sign up or log in at github.com.
2. Make a new repository: **+** (top right), **New repository**. Name it, for example `showshelf`, choose **Public**, and press **Create repository**.
3. On the new page, click the link **uploading an existing file**, drag in your `index.html`, and press **Commit changes**.
4. Open **Settings**, then **Pages** (left menu). Under "Build and deployment", choose **Deploy from a branch**, then the branch `main` and the folder `/ (root)`, and press **Save**.
5. After a minute or two, your site is at `https://USERNAME.github.io/REPO/` (your user name, then the repository's name). Reload the Pages settings to see the link.

Then: open your site, copy its address from the address bar, and paste it on the first line of the box.

COMMON MISTAKES

- Dropping the `index.html` file instead of the folder on Netlify Drop, or a folder where the file is called `index (1).html`.
- Pasting the repository address (`github.com/...`) instead of the site address (`....github.io/...`).
- A private repository: GitHub Pages needs it to be public on a free account.

NOTE

Both options need an email address for the account. A student who doesn't want an account can skip this step and give the demo from the downloaded file (with internet). GitHub Pages can take a few minutes the first time; let students start step 10 while they wait. To update a site later: on Netlify, open the site and drag the folder onto its Deploys page; on GitHub, upload the new `index.html` into the same repository.

A CORRECT ANSWER

```
https://ana-novak-showshelf.netlify.app
```

Step 10. Prepare your demo ~10 min

Teaches: A short demo: what it does, what you're proud of, what's next.

TASK

In the box, write **three lines**, each at least a full sentence:

1. What your site does.
2. The part you're proudest of, and why.
3. What you'd add next if you had another week.

Then practise once, out loud, with the site open.

COMMON MISTAKES

- One long line with all three answers: press Enter between them.
- Planning to demo from the Lab instead of the real site.

NOTE

Keep demos to 2 minutes with a visible timer. Have a backup: if the network is slow, students can demo from the downloaded file or from the Lab preview.

A CORRECT ANSWER

```
Ana's Shelf finds any TV show and keeps a watchlist of the ones I want to see.
I'm proudest of the Surprise me button, because it helped me choose tonight's show.
Next I'd add a page for each actor, with all the shows they're in.
```

Extra 1. A dark theme button ~20 min

Teaches: `classList.toggle` switches a class; `localStorage` remembers the choice.

TASK

In extra.js:

1. Create a button with the text "Dark mode" and the class `chip`, and append it to the header's `.nav`.
2. On click: toggle the class `dark` on `document.body`, then save `"dark"` or `"light"` in `localStorage` under the key `"theme"`.
3. At the start (outside the listener): if the saved theme is `"dark"`, add the class `dark` to `document.body`.

NOTE

The dark colours are fixed in the page (the `body.dark` rule), so students only write the JavaScript. Their own `my-theme.css` colours still apply in light mode.

A CORRECT ANSWER

```
// ... unchanged ...
  countLine.textContent = "Shows loaded: " + state.shows.length;
}

// ---------- Extra: a dark theme button ----------
const themeButton = document.createElement("button");
themeButton.className = "chip";
themeButton.textContent = "Dark mode";
document.querySelector(".nav").append(themeButton);

// Start dark if that was the last choice
if (localStorage.getItem("theme") === "dark") {
  document.body.classList.add("dark");
}

themeButton.addEventListener("click", () => {
  document.body.classList.toggle("dark");
  if (document.body.classList.contains("dark")) {
    localStorage.setItem("theme", "dark");
  } else {
    localStorage.setItem("theme", "light");
  }
});
```

Extra 2. Clear My List ~15 min

Teaches: `confirm()` asks OK or Cancel and gives `true` or `false`.

TASK

In extra.js:

1. Create a button with the text "Clear My List" and the class `button-secondary`, and append it to the hero of the My List view: `document.querySelector("#mylist-view .hero")`.
2. On click: if `confirm(...)` says OK, set `state.myList` to an empty array, then call `saveMyList()` and `render()`.

Add a few shows, open **My List**, and try both Cancel and OK.

A CORRECT ANSWER

```
// ... unchanged ...
  countLine.textContent = "Shows loaded: " + state.shows.length;
}

// ---------- Extra: clear My List, after asking ----------
const clearButton = document.createElement("button");
clearButton.className = "button-secondary";
clearButton.textContent = "Clear My List";
document.querySelector("#mylist-view .hero").append(clearButton);

clearButton.addEventListener("click", () => {
  if (confirm("Remove every show from My List?")) {
    state.myList = [];
    saveMyList();
    render();
  }
});
```

Extra 3. An About section ~15 min

Teaches: Build a section in JavaScript: create elements, `append` them together, then add them to the page.

TASK

In extra.js:

1. Create a `section` with the class `show`.
2. Inside it, put an `h2` that says "About this site" and a `p` with a few sentences about you and how you built the site.
3. Append the section to `homeView`.

A CORRECT ANSWER

```
// ... unchanged ...
  countLine.textContent = "Shows loaded: " + state.shows.length;
}

// ---------- Extra: an About section ----------
const about = document.createElement("section");
about.className = "show";
const aboutTitle = document.createElement("h2");
aboutTitle.textContent = "About this site";
const aboutText = document.createElement("p");
aboutText.textContent = "I'm " + SITE.owner + ". I built this site in the Web Dev in 11 Days course,
starting from zero: HTML, CSS, JavaScript and the free TVmaze API.";
about.append(aboutTitle, aboutText);
homeView.append(about);
```